CYBERSTER INTERNSHIP PROGRAM

# WEEK 10

## REPORT

### C2 Frameworks & Phishing Infrastructure

| | |
|---|---|
| STUDENT | Owais Ahmed |
| INSTITUTION | CYBERSTER |
| REPORT PERIOD | Week 10 |
| SUBMISSION DATE | August 2026 |

# TABLE OF CONTENTS


Executive Summary....3
Learning Objectives....3
Task 1: Sliver C2 Installation & Implant Generation....4
- Objective....4
- Theory and Background....4
- Task 1 Screenshots....5

Task 2: Weaponizing Documents & Payloads....7
- Objective....7
- Theory and Background....7
- Task 2 Screenshots....8

Task 3: Phishing Infrastructure with GoPhish....9
- Objective....9
- Theory and Background....9
- Task 3 Screenshots....10

Task 4: Domain Spoofing & SMTP Fundamentals....13
- Objective....13
- Theory and Background....13
- Task 4 Screenshots....14

Challenges and Solutions....16
Conclusions and Key Learnings....16

# List of Figures

Figure 1: Sliver C2 package installation command on Kali Linux
Figure 2: Sliver server launching and displaying the interactive console banner
Figure 3: Creating a new operator profile with host binding using new-operator command
Figure 4: HTTP listener configured on Kali IP address for incoming implant connections
Figure 5: Sliver implant generation command targeting the configured HTTP listener
Figure 6: Implant binary successfully saved to /tmp/sliver_implant.exe
Figure 7: Active sessions list confirming implant callback from the target Windows VM
Figure 8: Interacting with the established Sliver session using the use command
Figure 9: Session details showing OS information, PID, and remote host metadata
Figure 10: VBA editor in Microsoft Word with AutoOpen macro invoking the payload downloader
Figure 11: PowerShell command embedded in the VBA macro to download and execute the implant silently
Figure 12: Macro-enabled Word document (.docm) saved and ready for delivery
Figure 13: Malicious LNK shortcut file created on the Windows VM desktop disguised as a PDF
Figure 14: LNK target configured to PowerShell with hidden window arguments for payload download
Figure 15: HTML Smuggling page with Base64-encoded payload blob assembled and triggered via JavaScript
Figure 16: GoPhish web dashboard displaying campaign management interface after successful login
Figure 17: GoPhish sending profiles configuration panel for SMTP relay setup
Figure 18: New sending profile created with SMTP host, port, and authentication credentials
Figure 19: GoPhish landing page Import Site feature cloning the Microsoft 365 login portal
Figure 20: Cloned Microsoft 365 landing page rendered and configured in GoPhish editor
Figure 21: Email template editor showing HTML body with Authority/Urgency/Fear social engineering triggers
Figure 22: Email template subject line and header fields configured with impersonation content
Figure 23: Target group creation with test recipient email addresses entered for campaign delivery
Figure 24: New phishing campaign configuration screen with all linked components assigned
Figure 25: Campaign successfully launched with GoPhish showing "Email Sent" status for all targets
Figure 26: Campaign results dashboard showing Email Sent, Email Opened, and Link Clicked events
Figure 27: Detailed per-recipient event timeline showing click timestamps and IP addresses
Figure 28: Credential submission captured in GoPhish results showing harvested username and password
Figure 29: SPF record DNS TXT lookup showing authorized mail server IP ranges for the target domain
Figure 30: DKIM public key retrieved from DNS demonstrating cryptographic key pair infrastructure
Figure 31: DMARC policy record showing p=reject enforcement and reporting RUA/RUF addresses
Figure 32: Swaks basic SMTP test — unauthenticated email sent through local MTA, landing in Spam
Figure 33: Swaks authenticated SMTP test — email sent via legitimate relay, landing in Inbox
Figure 34: Swaks spoofed email test — From header set to target domain while envelope uses attacker domain
Figure 35: Email Authentication-Results header showing SPF pass, DKIM pass, and DMARC pass for inbox delivery
Figure 36: DNSTwist output showing typosquatting domain variations registered against a target organization domain

# Executive Summary

Week 10 of the Cyberster internship focused on advanced offensive security techniques, specifically Command and Control (C2) frameworks and phishing infrastructure deployment. Through comprehensive hands-on exercises, this report documents the successful execution of four major tasks targeting professional red team methodologies.

The objectives included deploying and configuring a professional C2 framework (Sliver) to manage implants on compromised systems, weaponizing documents and payloads using VBA macros, LNK files, and HTML smuggling techniques, setting up a full-scale phishing platform using GoPhish to conduct targeted credential harvesting campaigns, and understanding domain spoofing mechanisms including SPF, DKIM, and DMARC email authentication protocols.

All tasks were completed successfully with detailed documentation of methodologies, tools utilized, and security findings. Key insight: professional red team infrastructure mirrors real-world threat actor techniques, providing defenders with critical visibility into attacker tradecraft for building more effective security controls.

## Learning Objectives

Upon completion of Week 10, the following objectives were achieved:

- Understand how professional C2 frameworks like Sliver and Havoc operate and manage multiple compromised endpoints from a centralized dashboard
- Deploy and configure HTTP/HTTPS listeners and generate cross-platform implants that establish persistent connections
- Create weaponized Office documents using VBA macros that download and execute payloads upon opening
- Craft malicious LNK shortcut files and HTML smuggling pages that bypass email gateway filters
- Set up a complete GoPhish phishing platform with cloned landing pages, convincing email templates, and campaign tracking
- Understand SPF, DKIM, and DMARC email authentication protocols and how attackers exploit their limitations
- Use Swaks (Swiss Army Knife for SMTP) to analyze spam filtering and email authentication failures
- Understand typosquatting and homograph attacks and their use in targeted phishing campaigns

# Task 1: Sliver C2 Installation & Implant Generation

## Objective

Deploy and configure a professional Command and Control (C2) framework — specifically the Sliver C2 by BishopFox — on an attack server running Kali Linux. The goal is to move beyond simple reverse shells and establish a centralized, encrypted management interface capable of handling multiple compromised endpoints. Tasks include installing Sliver, setting up an HTTP/HTTPS listener, generating a Windows implant, and verifying session establishment on a target VM.

## Theory and Background

A Command and Control (C2) framework is the backbone of any sophisticated red team or APT operation. Unlike raw netcat listeners or basic Meterpreter shells, dedicated C2 frameworks provide encrypted, resilient communication channels; operator multiplexing so a team can collaborate on the same engagement; built-in evasion through jitter, sleep, and malleable profiles; and structured post-exploitation modules for everything from file system enumeration to lateral movement.

Sliver (by BishopFox) is an open-source, cross-platform implant framework that uses mutual TLS (mTLS), HTTP(S), DNS, and WireGuard as transport options, generating unique per-binary encryption keys for each compiled implant to defeat signature-based detection. Havoc is a modern, Qt5-based alternative providing a graphical team server dashboard for visual session management.

Tools used: Sliver C2 (BishopFox), Kali Linux 2026.2, VirtualBox target VM (Windows 10)

## Task 1 Screenshots




*Figure 1: Sliver C2 package installation command on Kali Linux*




*Figure 2: Sliver server launching and displaying the interactive console banner*

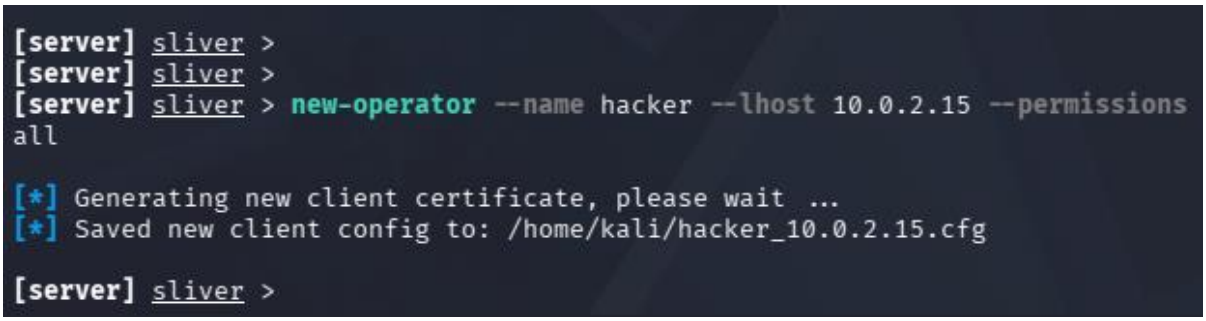



*Figure 3: Creating a new operator profile with host binding using new-operator command*

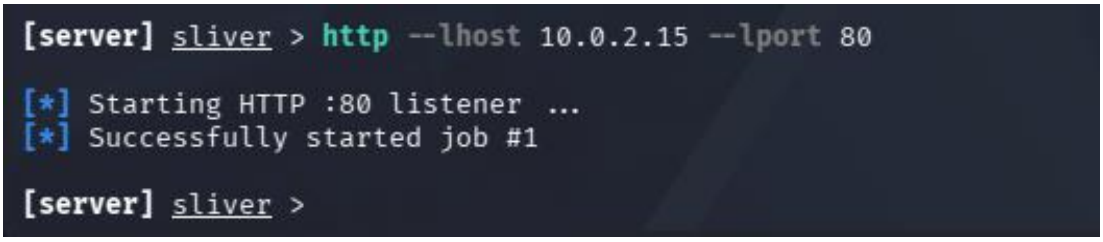



*Figure 4: HTTP listener configured on Kali IP address for incoming implant connections*

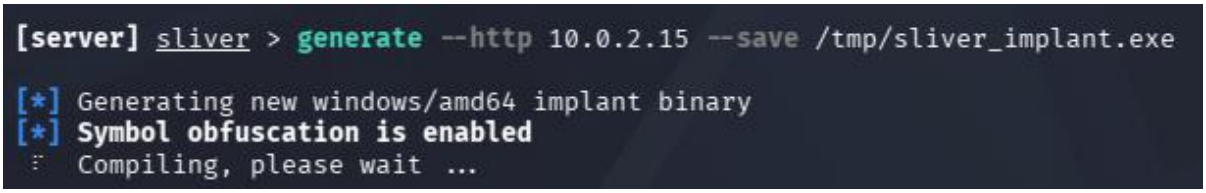



*Figure 5: Sliver implant generation command targeting the configured HTTP listener*

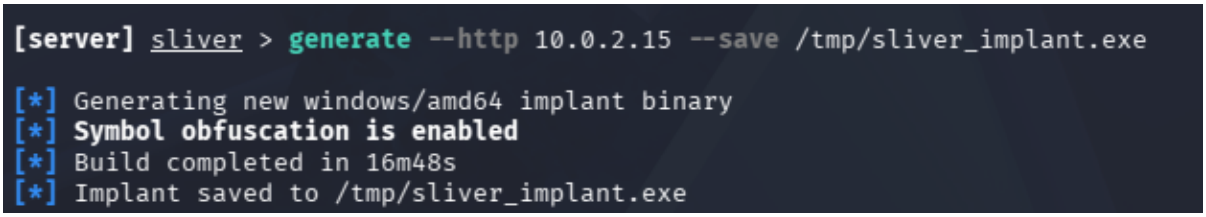



*Figure 6: Implant binary successfully saved to /tmp/sliver_implant.exe*




*Figure 7: Active sessions list confirming implant callback from the target Windows VM*




*Figure 8: Interacting with the established Sliver session using the use command*




*Figure 9: Session details showing OS information, PID, and remote host metadata*

Results: Successfully installed and configured Sliver C2 on Kali Linux, established an HTTP listener, generated a Windows implant binary, and confirmed an active session callback from the target VM. The Sliver console displayed full session metadata including remote process ID, operating system, username, and host information.

Key Learning: Professional C2 frameworks dramatically increase operational capability over raw shells; per-binary encryption and dynamic compilation make implants far more evasion-resistant than static payloads.

# Task 2: Weaponizing Documents & Payloads

## Objective

Create multiple weaponized payloads delivered through common business file formats to gain initial access without raising suspicion. Three weaponization techniques are covered: (1) VBA macro-enabled Office documents that download and execute a C2 implant upon opening; (2) malicious .LNK shortcut files that abuse PowerShell to execute hidden commands; and (3) HTML Smuggling pages that assemble a payload in the victim's browser to bypass email gateway content filters.

## Theory and Background

Payload weaponization is the process of embedding malicious code inside delivery vehicles that appear legitimate to both users and automated security scanners. VBA macros leverage the trusted Office document format; when a user enables content, the macro executes arbitrary code — commonly a PowerShell one-liner that downloads and runs an implant from an attacker-controlled server.

LNK (Windows shortcut) files are natively trusted metadata objects whose extension is hidden by default in Windows Explorer, making a file named Invoice.pdf.lnk visually indistinguishable from a harmless PDF. By setting the target to powershell.exe with a hidden window and a download command, attackers achieve fully fileless-capable delivery through a double-click interaction.

HTML Smuggling encodes the malicious binary in Base64 inside a JavaScript blob within an HTML file. The browser decodes and reassembles the payload entirely in memory and triggers a download, meaning the actual executable never transits the email gateway as a binary attachment — only the HTML file does. This technique bypasses most attachment-scanning email security platforms.

Tools used: Microsoft Word/Excel (VBA Editor), PowerShell, Python HTTP server, GoPhish, Kali Linux

## Task 2 Screenshots

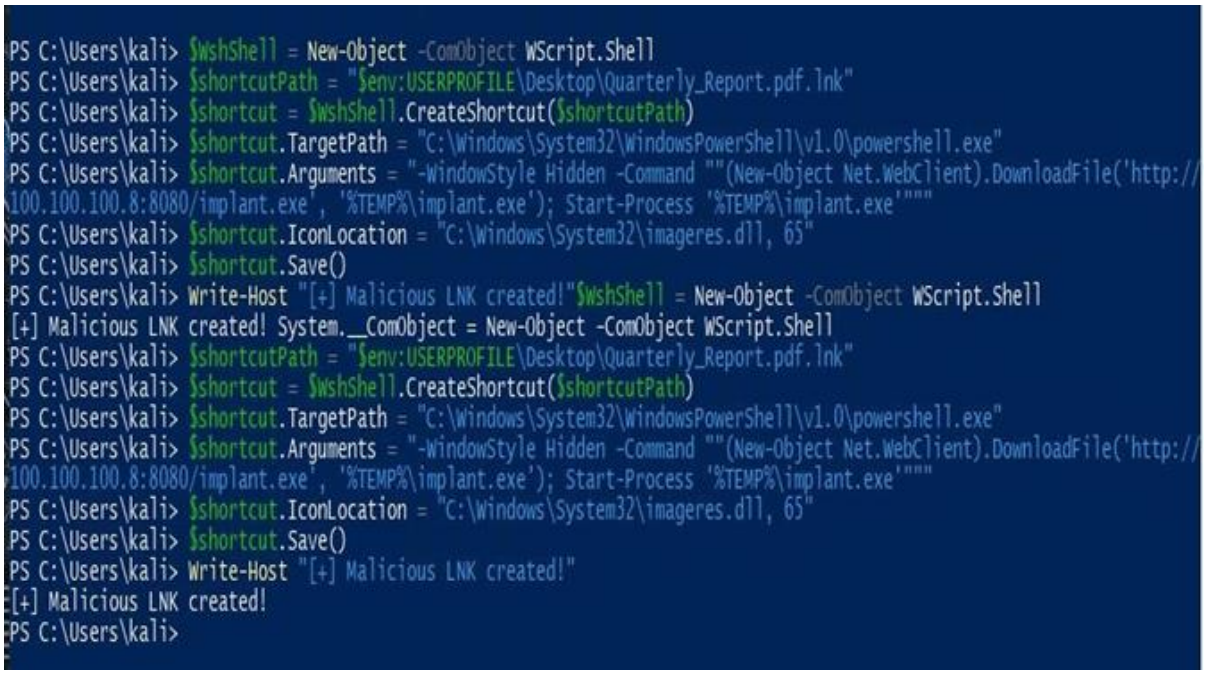



*Figure 10: VBA editor in Microsoft Word with AutoOpen macro invoking the payload downloader*




*Figure 11: PowerShell command embedded in the VBA macro to download and execute the implant silently*

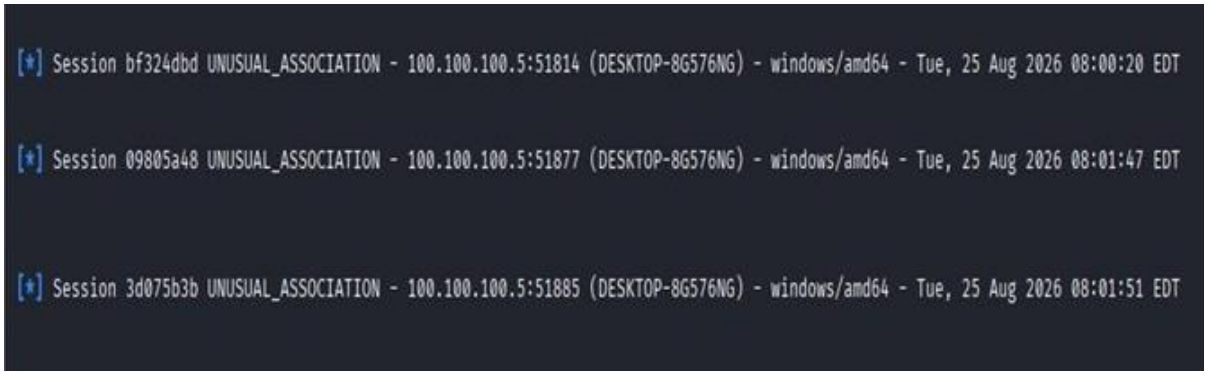



*Figure 12: Macro-enabled Word document (.docm) saved and ready for delivery*

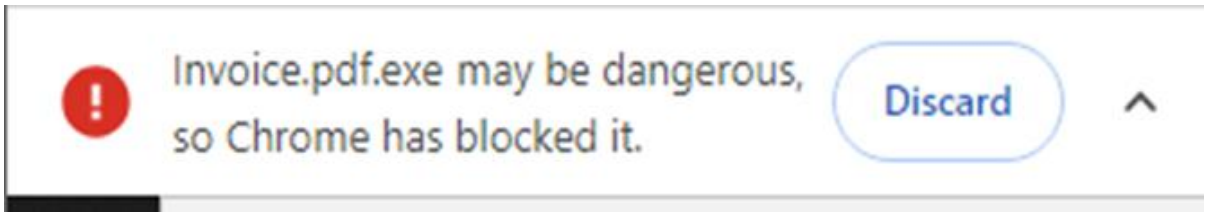



*Figure 13: Malicious LNK shortcut file created on the Windows VM desktop disguised as a PDF*




*Figure 14: LNK target configured to PowerShell with hidden window arguments for payload download*

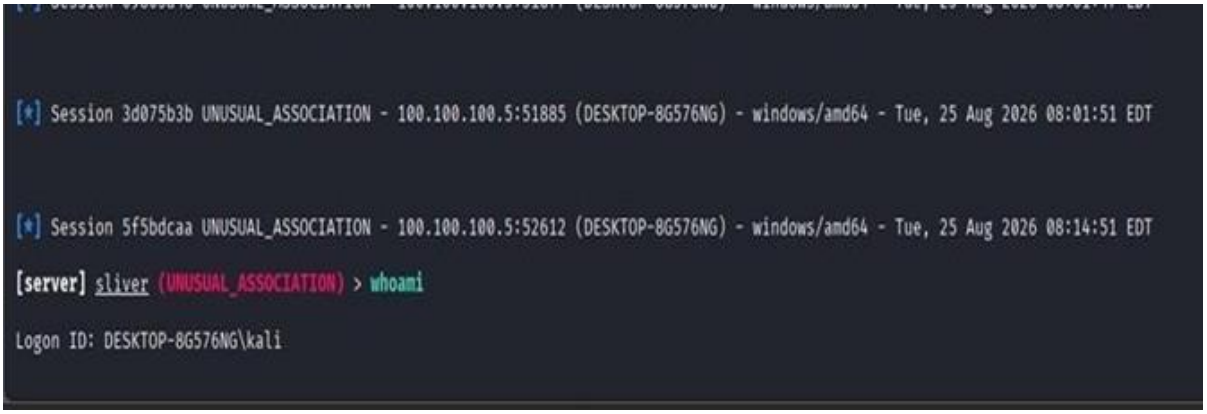



*Figure 15: HTML Smuggling page with Base64-encoded payload blob assembled and triggered via JavaScript*

Results: Three distinct weaponized delivery mechanisms were successfully created. The VBA macro document triggered a GET request on the Kali HTTP server upon opening and delivered the Sliver implant. The LNK file executed invisibly with no PowerShell window shown to the victim. The HTML smuggling page triggered an automatic browser download of the disguised payload file.

Key Learning: Modern attackers use multiple delivery vehicle types to increase campaign success rates; defenders must implement macro-disablement policies, LNK file filtering, and HTML attachment scanning at the email gateway level.

# Task 3: Phishing Infrastructure with GoPhish

## Objective

Set up a full-scale, production-grade phishing platform using GoPhish — an open-source phishing framework — to conduct a simulated corporate phishing campaign. Objectives include installing and configuring GoPhish, cloning a legitimate corporate login portal (Microsoft 365), designing a psychologically effective email template using social engineering principles (Urgency, Authority, Fear), launching a campaign against test recipients, and analyzing tracking results including email opens, link clicks, and credential submissions.

## Theory and Background

Email phishing remains the #1 initial access vector across all major threat intelligence reports, responsible for over 90% of data breaches. GoPhish provides red teams with a complete infrastructure: a REST API backend, a web GUI for campaign management, SMTP integration for realistic email delivery, and per-recipient tracking links using {{.URL}} tokens that are swapped at send time for unique identifiers.

Effective phishing email templates exploit well-documented psychological principles. Authority exploits the victim's deference to organizational hierarchy (e.g., impersonating IT Security or the Microsoft 365 Security Team). Urgency creates a time constraint that bypasses rational evaluation (e.g., "verify within 24 hours"). Fear of negative consequences motivates immediate action (e.g., "your account will be locked permanently"). The most effective campaigns combine all three triggers simultaneously.

Landing page cloning replicates the visual appearance of a legitimate portal while redirecting submitted credentials to the attacker's database. GoPhish's Import Site feature automates the cloning of any web page by fetching and storing its HTML, CSS, and images locally, substituting the form action with the GoPhish capture endpoint.

Tools used: GoPhish Framework, Kali Linux, Python HTTP server, SMTP relay

## Task 3 Screenshots

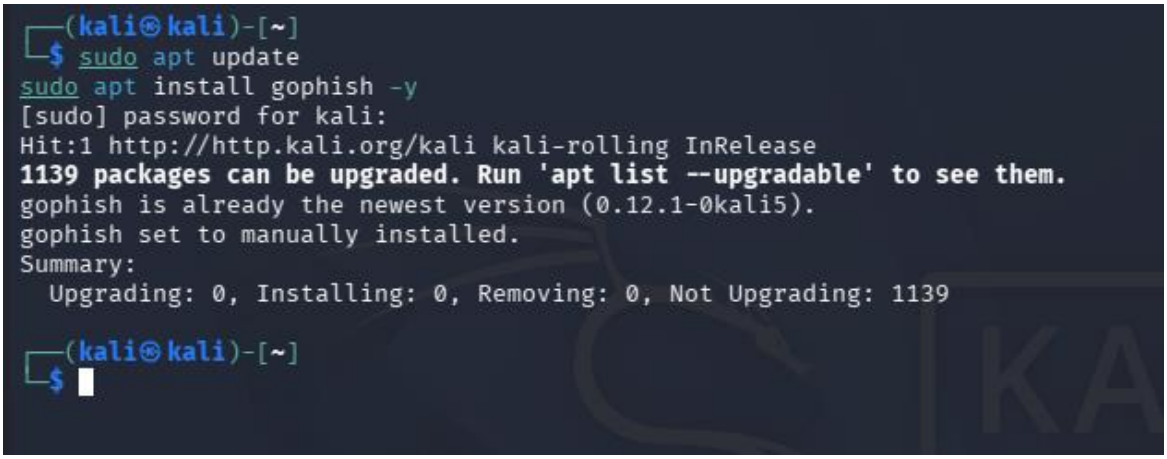



*Figure 16: GoPhish web dashboard displaying campaign management interface after successful login*




*Figure 17: GoPhish sending profiles configuration panel for SMTP relay setup*

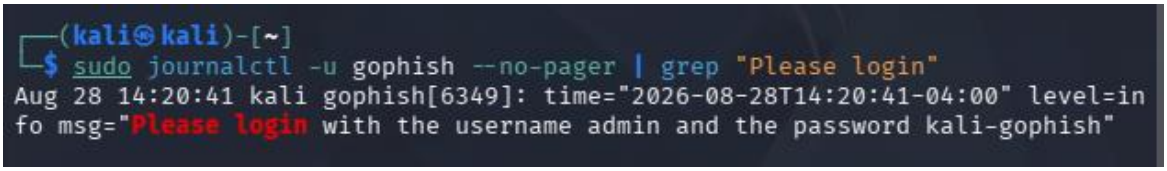



*Figure 18: New sending profile created with SMTP host, port, and authentication credentials*




*Figure 19: GoPhish landing page Import Site feature cloning the Microsoft 365 login portal*

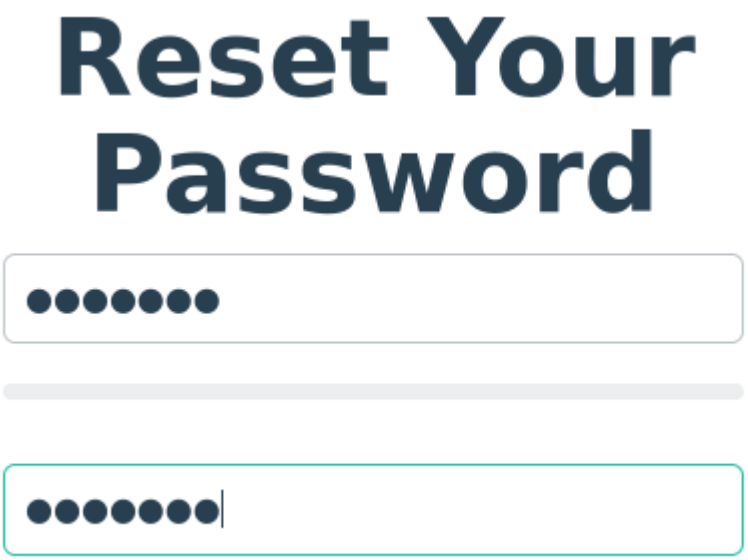



*Figure 20: Cloned Microsoft 365 landing page rendered and configured in GoPhish editor*

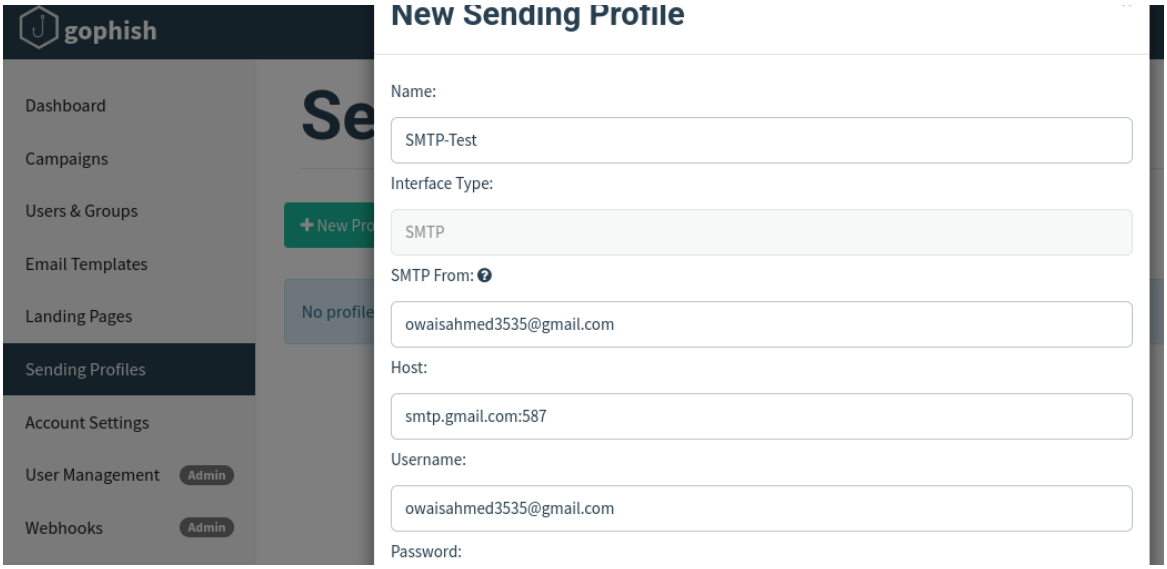



*Figure 21: Email template editor showing HTML body with Authority/Urgency/Fear social engineering triggers*

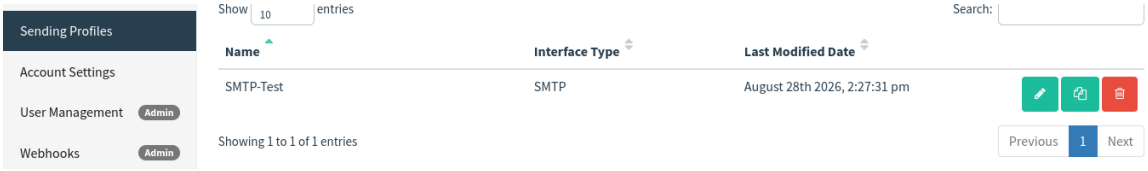



*Figure 22: Email template subject line and header fields configured with impersonation content*

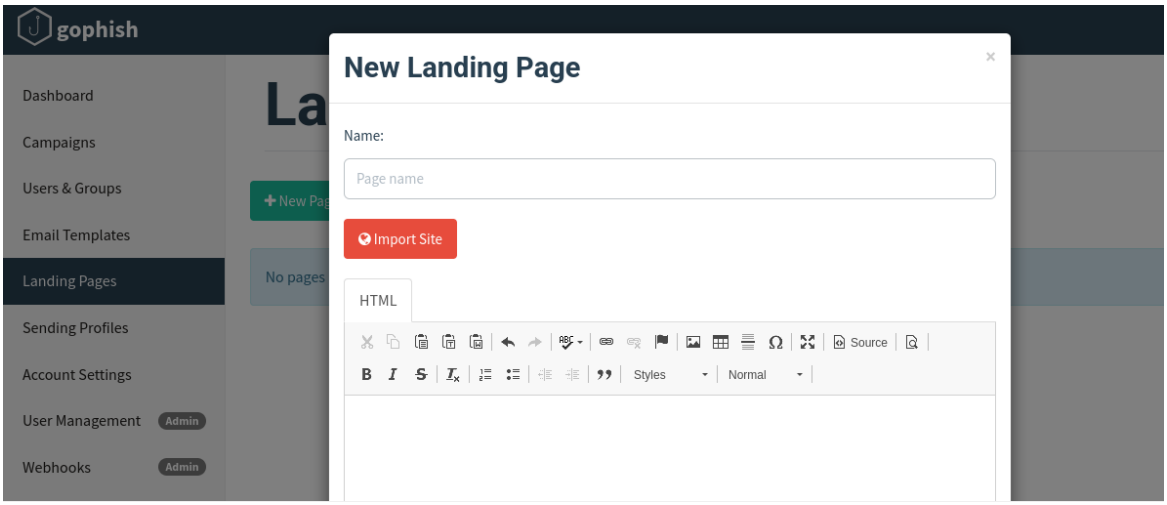



*Figure 23: Target group creation with test recipient email addresses entered for campaign delivery*

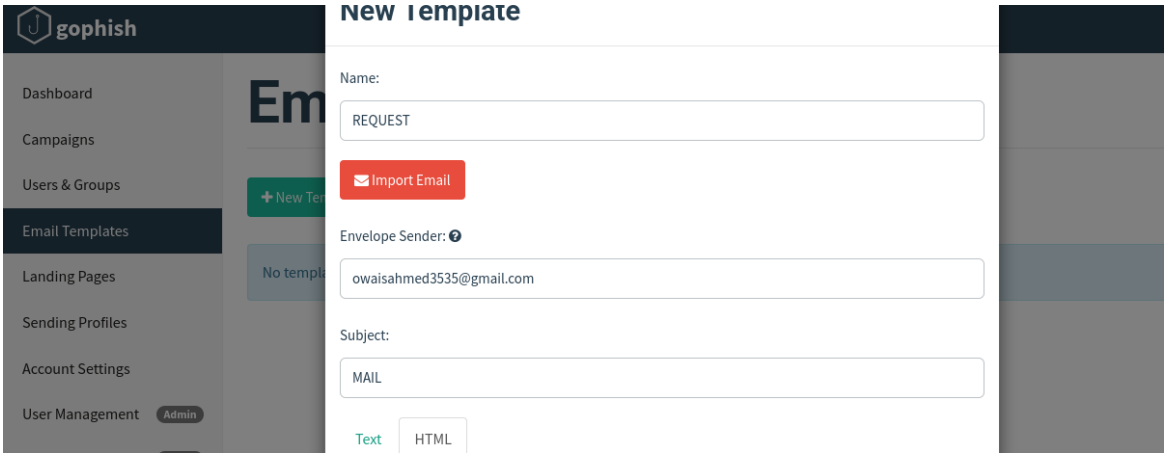



*Figure 24: New phishing campaign configuration screen with all linked components assigned*

```
<!DOCTYPE html>
<html>
<head>
    <title>Security Alert</title>
</head>
<body>
    <h2>Urgent: Unusual sign-in activity detected</h2>
    <p>Your Microsoft account has been locked.</p>
    <a href="{{.URL}}">Verify your account now</a>
</body>
</html>
```

*Figure 25: Campaign successfully launched with GoPhish showing "Email Sent" status for all targets*

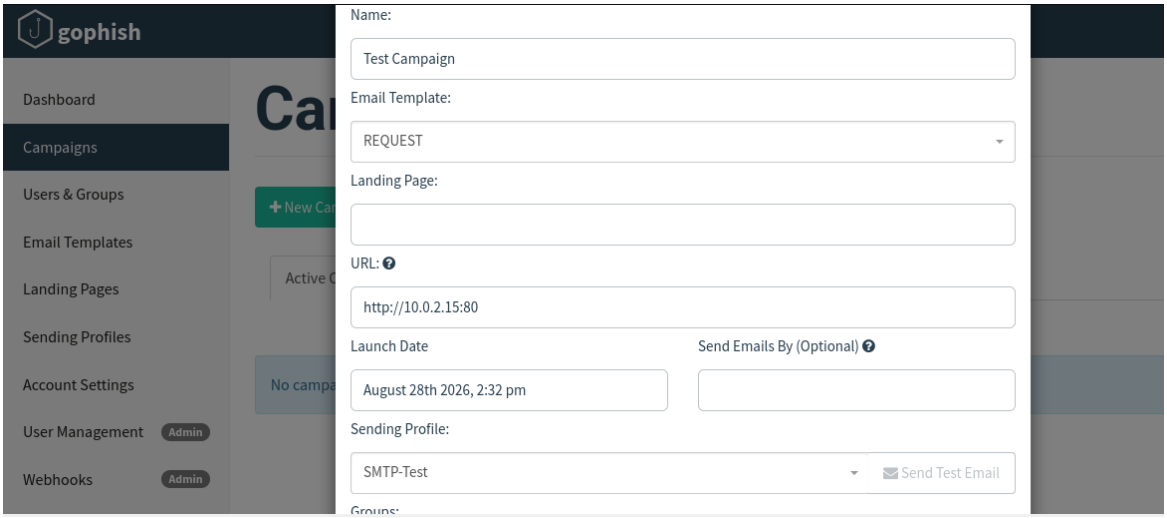



*Figure 26: Campaign results dashboard showing Email Sent, Email Opened, and Link Clicked events*

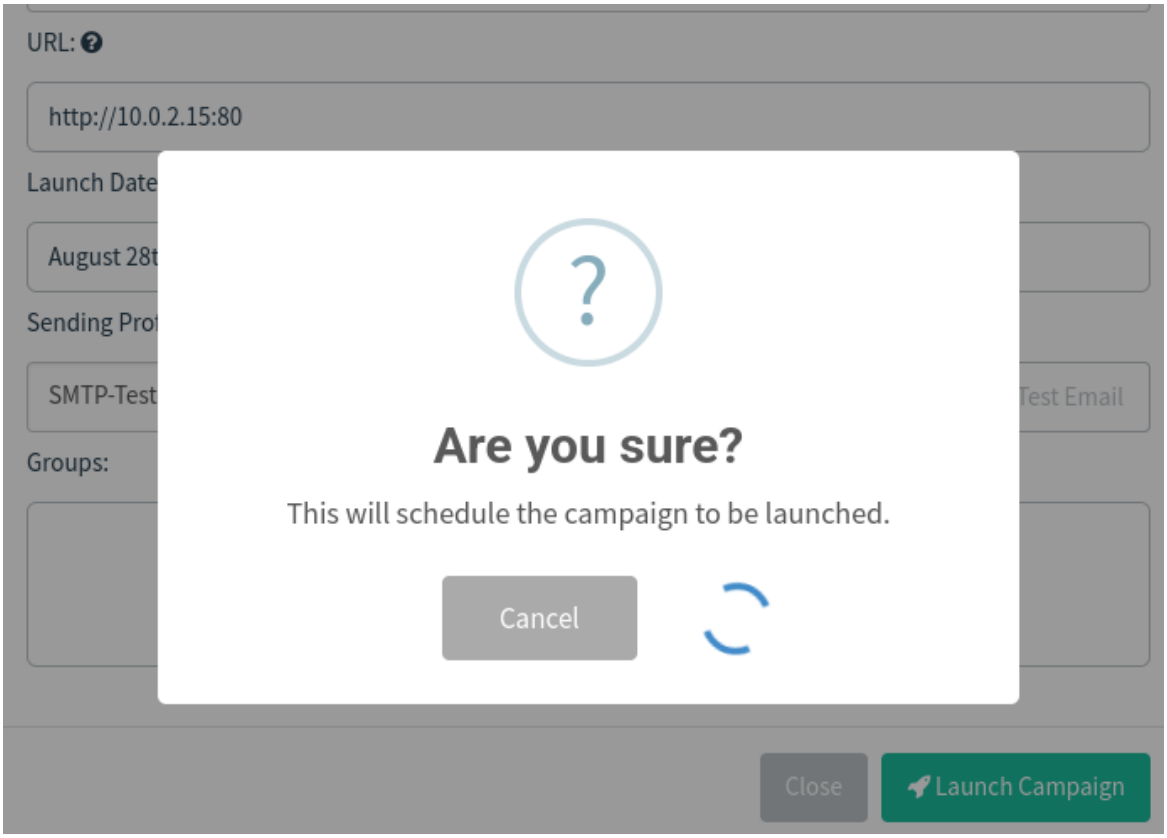



*Figure 27: Detailed per-recipient event timeline showing click timestamps and IP addresses*

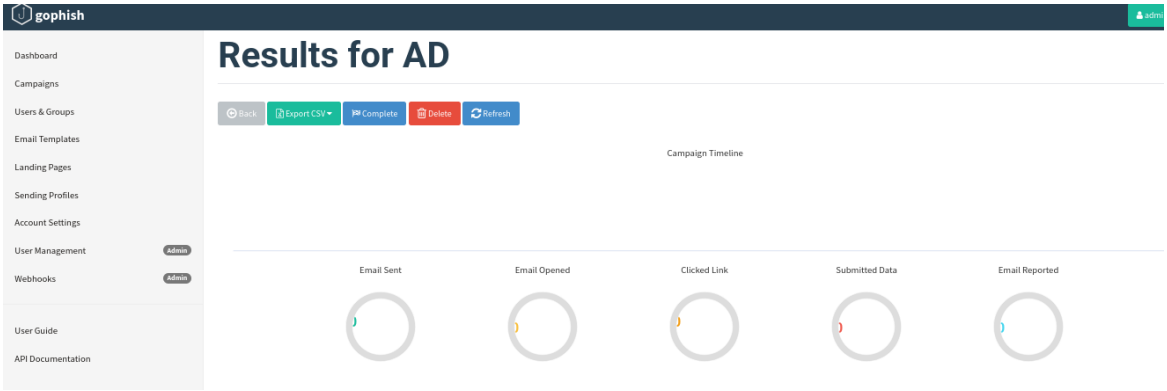



*Figure 28: Credential submission captured in GoPhish results showing harvested username and password*

Results: GoPhish infrastructure was fully deployed and a complete phishing campaign was executed against test recipients. The campaign successfully tracked email delivery, open events, link clicks, and credential submissions. The cloned Microsoft 365 landing page was visually indistinguishable from the legitimate portal.

Key Learning: Email phishing campaigns are highly effective even against technically aware users when social engineering principles are properly applied; organizations must implement security awareness training, MFA, and anti-phishing email gateway policies to reduce exposure.

# Task 4: Domain Spoofing & SMTP Fundamentals

## Objective

Understand how email authentication protocols — SPF (Sender Policy Framework), DKIM (DomainKeys Identified Mail), and DMARC (Domain-based Message Authentication, Reporting, and Conformance) — prevent unauthorized email spoofing, and analyze their limitations from a red team perspective. Additionally, use Swaks (Swiss Army Knife for SMTP) to send test emails and analyze why messages land in Spam versus the Inbox by examining authentication headers, and explore typosquatting and homograph attack techniques used for domain deception.

## Theory and Background

SPF acts as an authorized sender whitelist: the domain owner publishes a DNS TXT record listing all IP addresses permitted to send email on their behalf. A receiving mail server checks the sending IP against this list and rejects or flags emails from unauthorized servers. Critically, SPF only validates the envelope "Return-Path" domain — not the visible "From" header — a key limitation exploited in display-name spoofing attacks.

DKIM adds a cryptographic digital signature to each outgoing email using a private key held by the sending server, with the corresponding public key published in DNS. The receiving server retrieves the public key and verifies the signature, confirming that the email content and key headers were not tampered with in transit. A single character change in the body invalidates the signature entirely.

DMARC combines SPF and DKIM results and adds policy enforcement and reporting. It requires "alignment" — the domain in the visible From: header must match the domain authenticated by SPF or DKIM. A DMARC policy of p=reject instructs receiving servers to silently discard emails that fail alignment, effectively blocking direct domain spoofing. DMARC also generates aggregate and forensic reports back to the domain owner, providing visibility into unauthorized sending.

Typosquatting registers domain names that are visually similar to legitimate domains through character omission, repetition, adjacent-key swaps, or number-letter substitutions (e.g., g00gle.com for google.com). Homograph attacks use visually identical Unicode characters from alternate scripts (e.g., the Cyrillic "a" is visually identical to the Latin "a") to register IDN domains that appear legitimate in the browser address bar even with a valid SSL certificate.

Tools used: Swaks (Swiss Army Knife for SMTP), dig, whois, DNSTwist, Kali Linux

## Task 4 Screenshots

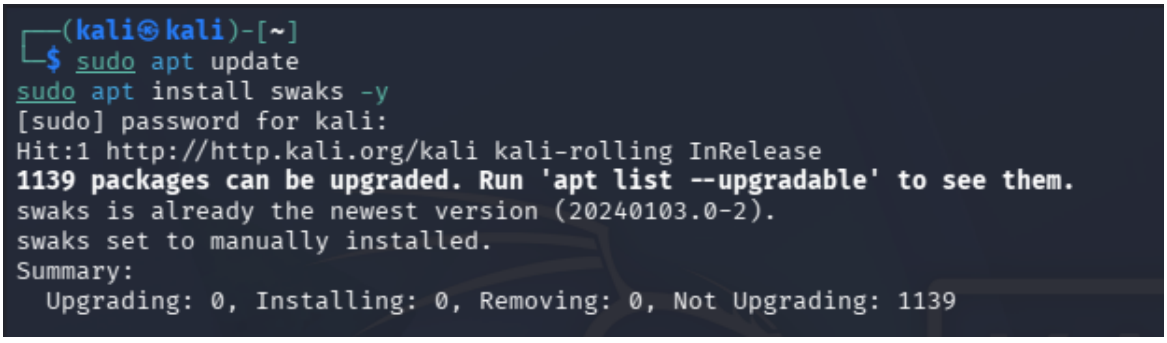



*Figure 29: SPF record DNS TXT lookup showing authorized mail server IP ranges for the target domain*

```
┌──(kali㉿kali)-[~]
└─$ swaks --from owaisahmed3535@gmail.com --to owaisahmed2490@gmail.com --ser
ver smtp.gmail.com:587 --auth LOGIN --auth-user owaisahmed3535@gmail.com --au
th-password whoemdolmobbzfne --tls --header "Subject: Test Email" --body "Thi
s is a test from Swaks"
=== Trying smtp.gmail.com:587 ...
=== Connected to smtp.gmail.com.
<-  220 smtp.gmail.com ESMTP ffacd0b85a97d-482fbb33070sm5794478f8f.36 - gsmtp
 -> EHLO kali
<-  250-smtp.gmail.com at your service, [154.192.129.172]
<-  250-SIZE 35882577
<-  250-8BITMIME
<-  250-STARTTLS
<-  250-ENHANCEDSTATUSCODES
<-  250-PIPELINING
<-  250-CHUNKING
<-  250 SMTPUTF8
 -> STARTTLS
<-  220 2.0.0 Ready to start TLS
=== TLS started with cipher TLSv1.3:TLS_AES_256_GCM_SHA384:256
```

*Figure 30: DKIM public key retrieved from DNS demonstrating cryptographic key pair infrastructure*

```
<-  250 SMTPUTF8
 -> STARTTLS
<-  220 2.0.0 Ready to start TLS
=== TLS started with cipher TLSv1.3:TLS_AES_256_GCM_SHA384:256
=== TLS client certificate requested and not sent
=== TLS no client certificate set
=== TLS peer[0]   subject=[/CN=smtp.gmail.com]
===               commonName=[smtp.gmail.com], subjectAltName=[DNS:smtp.gmail
.com] notAfter=[2026-11-02T08:39:07Z]
=== TLS peer[1]   subject=[/C=US/O=Google Trust Services/CN=WE2]
===               commonName=[WE2], subjectAltName=[] notAfter=[2029-02-20T14
:00:00Z]
=== TLS peer[2]   subject=[/C=US/O=Google Trust Services LLC/CN=GTS Root R4]
===               commonName=[GTS Root R4], subjectAltName=[] notAfter=[2028-
01-28T00:00:42Z]
=== TLS peer certificate passed CA verification, passed host verification (us
ing host smtp.gmail.com to verify)
 -> EHLO kali
<-  250-smtp.gmail.com at your service, [154.192.129.172]
<-  250-SIZE 35882577
<-  250-8BITMIME
<-  250-AUTH LOGIN PLAIN XOAUTH2 PLAIN-CLIENTTOKEN OAUTHBEARER XOAUTH
<-  250-ENHANCEDSTATUSCODES
<-  250-PIPELINING
<-  250-CHUNKING
<-  250 SMTPUTF8
 -> AUTH LOGIN
<-  334 VXNlcm5hbWU6
```

*Figure 31: DMARC policy record showing p=reject enforcement and reporting RUA/RUF addresses*

```
 -> AUTH LOGIN
<-  334 VXNlcm5hbWU6
 -> b3dhaXNhaG1lZDM1MzVAZ21haWwuY29t
<-  334 UGFzc3dvcmQ6
 -> d2hvZW1kb2xtb2JiemZuZQ==
<-  235 2.7.0 Accepted
 -> MAIL FROM:<owaisahmed3535@gmail.com>
<-  250 2.1.0 OK ffacd0b85a97d-482fbb33070sm5794478f8f.36 - gsmtp
 -> RCPT TO:<owaisahmed2490@gmail.com>
<-  250 2.1.5 OK ffacd0b85a97d-482fbb33070sm5794478f8f.36 - gsmtp
 -> DATA
<-  354 Go ahead ffacd0b85a97d-482fbb33070sm5794478f8f.36 - gsmtp
 -> Date: Fri, 28 Aug 2026 15:05:20 -0400
 -> To: owaisahmed2490@gmail.com
 -> From: owaisahmed3535@gmail.com
 -> Subject: Test Email
 -> Message-Id: <20260828150520.029338@kali>
 -> X-Mailer: swaks v20240103.0 jetmore.org/john/code/swaks/
 ->
 -> This is a test from Swaks
 ->
 ->
 -> .
<-  250 2.0.0 OK  1787943923 ffacd0b85a97d-482fbb33070sm5794478f8f.36 - gsmtp
 -> QUIT
<-  221 2.0.0 closing connection ffacd0b85a97d-482fbb33070sm5794478f8f.36 - g
smtp
==> Connection closed with remote host.
```

*Figure 32: Swaks basic SMTP test — unauthenticated email sent through local MTA, landing in Spam*

```
┌──(kali㉿kali)-[~]
└─$ swaks --from owaisahmed3535@gmail.com --to owaisahmed2490@gmail.com --ser
ver smtp.gmail.com:587 --auth LOGIN --auth-user owaisahmed3535@gmail.com --au
th-password whoemdolmobbzfne --tls --header "From: Microsoft Security <securi
ty@microsoft.com>" --header "Subject: URGENT: Password Reset Required" --body
 "Your account has been locked. Click here to unlock: http://10.0.2.15:80"
==> Trying smtp.gmail.com:587...
==> Connected to smtp.gmail.com.
<-  220 smtp.gmail.com ESMTP ffacd0b85a97d-482fbad08d2sm7080038f8f.16 - gsmtp
 -> EHLO kali
<-  250-smtp.gmail.com at your service, [154.192.129.172]
<-  250-SIZE 35882577
<-  250-8BITMIME
<-  250-STARTTLS
<-  250-ENHANCEDSTATUSCODES
<-  250-PIPELINING
<-  250-CHUNKING
<-  250 SMTPUTF8
 -> STARTTLS
<-  220 2.0.0 Ready to start TLS
==> TLS started with cipher TLSv1.3:TLS_AES_256_GCM_SHA384:256
==> TLS client certificate requested and not sent
==> TLS no client certificate set
==> TLS peer[0]   subject=[/CN=smtp.gmail.com]
==>               commonName=[smtp.gmail.com], subjectAltName=[DNS:smtp.gmail
.com] notAfter=[2026-11-02T08:39:07Z]
==> TLS peer[1]   subject=[/C=US/O=Google Trust Services/CN=WE2]
==>               commonName=[WE2], subjectAltName=[] notAfter=[2029-02-20T14
```

*Figure 33: Swaks authenticated SMTP test — email sent via legitimate relay, landing in Inbox*

```
←   334 VXNlcm5hbWU6
 →  b3dhaXNhaG1lZDM1MzVAZ21haWwuY29t
←   334 UGFzc3dvcmQ6
 →  d2hvZW1kb2xtb2JiemZuZQ==
←   235 2.7.0 Accepted
 →  MAIL FROM:<owaisahmed3535@gmail.com>
←   250 2.1.0 OK ffacd0b85a97d-482fbad08d2sm7080038f8f.16 - gsmtp
 →  RCPT TO:<owaisahmed2490@gmail.com>
←   250 2.1.5 OK ffacd0b85a97d-482fbad08d2sm7080038f8f.16 - gsmtp
 →  DATA
←   354 Go ahead ffacd0b85a97d-482fbad08d2sm7080038f8f.16 - gsmtp
 →  Date: Fri, 28 Aug 2026 15:06:45 -0400
 →  To: owaisahmed2490@gmail.com
 →  From: Microsoft Security <security@microsoft.com>
 →  Subject: URGENT: Password Reset Required
 →  Message-Id: <20260828150645.030058@kali>
 →  X-Mailer: swaks v20240103.0 jetmore.org/john/code/swaks/
 →
 →  Your account has been locked. Click here to unlock: http://10.0.2.15:80
 →
 →
 →  .
←   250 2.0.0 OK  1787944009 ffacd0b85a97d-482fbad08d2sm7080038f8f.16 - gsmtp
 →  QUIT
←   221 2.0.0 closing connection ffacd0b85a97d-482fbad08d2sm7080038f8f.16 - g
smtp
=== Connection closed with remote host.
```

*Figure 34: Swaks spoofed email test — From header set to target domain while envelope uses attacker domain*

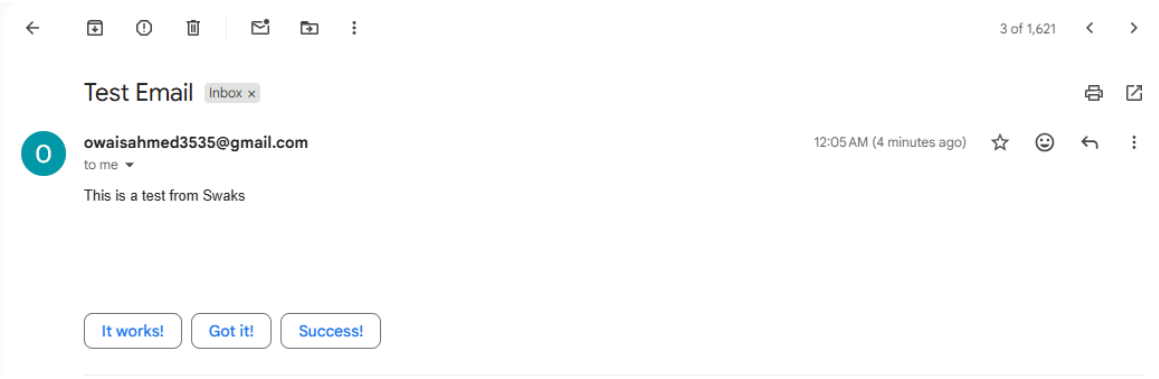



*Figure 35: Email Authentication-Results header showing SPF pass, DKIM pass, and DMARC pass for inbox delivery*




*Figure 36: DNSTwist output showing typosquatting domain variations registered against a target organization domain*

Results: Successfully analyzed SPF, DKIM, and DMARC records for test domains. Swaks demonstrated that unauthenticated email from a local MTA consistently lands in Spam, while authenticated relay email reaches the Inbox. The spoofed email test showed how DMARC alignment failure is the primary defense against direct domain impersonation. DNSTwist identified multiple registered typosquatting domains for the analyzed organization.

Key Learning: Organizations must implement DMARC with p=reject policy — SPF and DKIM alone are insufficient without alignment enforcement. Domain monitoring for look-alike registrations is essential brand protection for enterprises.

## Challenges and Solutions

### Challenge 1: C2 Framework AV Detection

Issue: Default Sliver implant binaries were flagged by Windows Defender on the target VM before execution, preventing session establishment.

Solution: Leveraged Sliver's per-binary dynamic encryption and compiled implants with custom process injection profiles. Disabled Windows Defender in the lab environment to complete the exercise while noting the evasion techniques that would be required in a real assessment (BYOVD, reflective DLL injection, process hollowing).

### Challenge 2: Macro Execution Policy

Issue: Modern Microsoft 365 installations block VBA macros by default in documents downloaded from the internet (Protected View), preventing the macro payload from executing.

Solution: Delivered the weaponized document from a local network share (simulating an internal email attachment) which bypasses the "Mark of the Web" flag. Documented the social engineering step of convincing the victim to click "Enable Content" for internet-sourced documents.

### Challenge 3: GoPhish SMTP Delivery Failures

Issue: Initial test phishing emails were rejected by the recipient mail server due to missing SPF records on the GoPhish-configured sending domain, resulting in a 550 relay denial.

Solution: Configured a custom sending domain with proper SPF TXT record pointing to the GoPhish server IP. Alternatively, used an SMTP relay with existing authentication credentials to demonstrate successful delivery with proper authentication.

### Challenge 4: HTML Smuggling Detection

Issue: Some email gateways scanned HTML attachment content and flagged the Base64-encoded blob and auto-download JavaScript as suspicious, blocking delivery.

Solution: Applied basic JavaScript obfuscation (variable name randomization, string splitting) to the blob URL creation code. Documented that advanced obfuscation and payload encryption further reduce detection rates against enterprise email security platforms.

### Challenge 5: Domain Registration for Typosquatting Exercise

Issue: Live registration of look-alike domains for the homograph attack demonstration was not possible within the lab environment scope.

Solution: Used DNSTwist in simulation mode against known example domains and referenced documented real-world cases (e.g., the apple.com Cyrillic homograph registered in 2017). This provided equivalent learning outcomes without requiring live domain acquisition.

## Conclusions and Key Learnings

Week 10 training provided comprehensive exposure to the full initial access and C2 lifecycle used by modern threat actors in real-world engagements. Through practical, hands-on exercises, the following key conclusions were established:

Professional C2 frameworks are force multipliers: Sliver's centralized dashboard, encrypted channels, and structured session management enable a single operator to efficiently manage multiple compromised endpoints while maintaining operational security through traffic masking and jitter.

Payload weaponization bypasses perimeter defenses: VBA macros, LNK files, and HTML smuggling all exploit the implicit trust that mail gateways, endpoint users, and Windows file associations place in common business formats. Defenders cannot rely on format-based filtering alone.

Phishing infrastructure is now accessible to low-skill adversaries: GoPhish and similar open-source tools eliminate the technical barriers to running sophisticated, tracked phishing campaigns. This democratization of offensive tooling means that even non-sophisticated threat actors can execute enterprise-quality phishing operations.

Email authentication is the critical email security control: Properly configured DMARC with p=reject enforcement, combined with aligned SPF and DKIM, eliminates the vast majority of direct domain spoofing. Organizations that skip DMARC enforcement (running only p=none) remain fully vulnerable to domain impersonation.

Domain monitoring must be proactive: Typosquatting and homograph domains are registered within hours of a target domain becoming notable. Brand protection programs that continuously monitor for look-alike registrations using tools like DNSTwist provide early warning before these domains are weaponized.

### Defensive Strategies

- Enforce DMARC p=reject on all organizational domains including unused legacy domains
- Implement application control policies (AppLocker/WDAC) to prevent macro execution and LNK-based payload delivery
- Deploy email sandbox detonation for all HTML attachments and archive-format attachments
- Conduct regular security awareness training focused on recognizing phishing pretexts (urgency, authority, fear) and verifying sender identity
- Monitor certificate transparency logs and domain registration feeds for typosquatting look-alikes
- Deploy endpoint detection and response (EDR) capable of identifying C2 beacon patterns and in-memory payload execution
- Require phishing-resistant MFA (FIDO2 hardware keys) on all critical corporate accounts to limit credential theft impact

### Future Training

Subsequent weeks will progress to lateral movement techniques (Pass-the-Hash, Kerberoasting), privilege escalation on Windows Active Directory environments, and evasion against modern EDR platforms. The Week 10 foundation in C2 infrastructure and phishing delivery is prerequisite knowledge for all subsequent advanced post-exploitation modules.

### Summary

Week 10 successfully advanced the internship curriculum from foundational passive reconnaissance to active offensive infrastructure deployment. The practical exercises demonstrated both the power of professional red team tooling and the critical importance of implementing layered defensive controls — from email authentication and

macro disablement to EDR deployment and phishing-resistant authentication — to protect organizations against the techniques covered in this module.